Họ và tên: Nguyễn Thị Ngọc Huyền
MSSV: 24520714
Lớp: SE373.R11
Link GitHub: https://github.com/quackii00/BTH7_Challenge_A_persistent_memory

BÁO CÁO BTH7 – CHALLENGE A: PERSISTENT MEMORY

1. Bài toán và mục tiêu

Trong các ứng dụng thực tế kéo dài nhiều ngày, mỗi lần gọi LLM API bản chất là stateless (chỉ nhìn thấy dữ liệu của request hiện tại). Khi người dùng tắt chương trình và mở một Session mới, toàn bộ lịch sử trò chuyện (messages) của phiên trước không còn trong ngữ cảnh. Do đó, cần xây dựng hệ thống Agent Harness tích hợp Persistent Memory trên ổ đĩa để lưu trữ, truy xuất (Retrieval), tiêm vào ngữ cảnh (Injection) và cập nhật khi quy ước dự án thay đổi.

Mục tiêu cần chứng minh:

- Kế thừa kiến trúc Agent và Harness đã có từ BTH3
- Chứng minh 1 (Nhớ quy ước qua session mới): Khi mở một session mới hoàn toàn (lịch sử hội thoại trống rỗng), Agent tự động đọc bộ nhớ từ ổ đĩa, nạp vào Context và áp dụng đúng các quy ước dự án đã thống nhất ở session trước.
- Chứng minh 2 (Cập nhật khi quy ước thay đổi): Khi người dùng thay đổi quy ước dự án, hệ thống giải quyết xung đột bằng cách đánh dấu quy ước cũ đã bị thay thế (lưu nguồn và thời điểm), cập nhật quy ước mới làm hiện hành, và các session tiếp theo sử dụng đúng quy ước mới.

Kịch bản kiểm chứng được thiết kế qua 4 Sessions độc lập:

Session	Yêu cầu đầu vào (User)	Kỳ vọng ở Agent & Harness
Session 1	Thống nhất quy ước ban đầu: - stack: PostgreSQL, TypeScript, REST API camelCase - auth: Sử dụng JWT	Agent gọi tool save_convention_safe lưu bền vững 2 quy ước xuống ổ đĩa (MEMORY.md, memory/stack.md, memory/auth.md).
Session 2 (Chứng minh 1)	Mở session mới (lịch sử chat rỗng). Hỏi dự án đang dùng stack và authentication nào?	Harness đọc MEMORY.md từ ổ đĩa và inject vào Context. Agent trả lời đúng PostgreSQL, TypeScript, REST API camelCase và JWT.
Session 3	Đổi quy ước auth: Chuyển từ JWT sang Session Cookie (HTTP-only) để tránh rủi ro XSS.	Agent gọi tool cập nhật chủ đề auth. Hệ thống đánh dấu JWT đã bị thay thế (lưu nguồn session_1 và thời điểm) và đặt Session Cookie làm hiện hành.

Session 4 (Chứng minh 2)	Mở session mới sau khi đổi quy ước. Kiểm tra quy ước hiện hành và lịch sử đã thay thế.	Harness inject MEMORY.md mới. Agent gọi read_topic_detail_safe('auth') đọc memory/auth.md, xác nhận hiện hành là Session Cookie và JWT đã bị thay thế.

2. Thiết kế và cài đặt các lớp Harness & Persistent Memory

2.1. Tổ chức lưu trữ Persistent Memory (MemoryStore)

Để tránh hiện tượng Context Bloat thì lớp MemoryStore trong memory_store.py tổ chức bộ nhớ trên ổ đĩa theo mô hình Index ngắn và Topic files chi tiết:

- MEMORY.md (Index ngắn): Chỉ chứa các quy ước hiện hành (Active) của từng chủ đề. Luôn được Harness nạp vào Context đầu mỗi session.
- memory/<topic>.md (memory/stack.md, memory/auth.md): Chứa chi tiết quy định hiện hành và toàn bộ lịch sử các quy định đã bị thay thế (Superseded) kèm nguồn và thời điểm. Chỉ được nạp khi Agent gọi tool tra cứu chi tiết.
- memory_store.json: Lưu cấu trúc dữ liệu gốc dạng JSON để xử lý xung đột bằng code.

2.2. Giải quyết xung đột khi quy ước thay đổi (upsert_convention)

Khi quy ước của một chủ đề đã tồn tại bị thay đổi, hệ thống không chỉ append (gây mâu thuẫn giữa quy định cũ và mới) và cũng không xóa trắng lịch sử, mà chuyển quy ước cũ vào danh sách superseded_history kèm nguồn và thời điểm:

```
superseded_entry = {
    "rule": old_rule,
    "source_session": current["source_session"],
    "created_at": current["updated_at"],
    "superseded_at": now_str,
    "superseded_by_session": session_id,
    "superseded_reason": reason
}
current.setdefault("superseded_history", []).append(superseded_entry)

# Cập nhật quy ước hiện hành mới
current["active_rule"] = rule
current["reason"] = reason
current["source_session"] = session_id
current["updated_at"] = now_str
action_type = "updated_and_superseded_old"
```

2.3. Cơ chế Retrieval & Injection trong Harness

Mỗi khi bắt đầu một Session mới, Harness reset lại StepBudget và LoopDetector của phiên, đồng thời thực hiện bước Retrieval (đọc MEMORY.md từ ổ đĩa) và Injection (đưa vào System Prompt của Agent):

```
def get_injected_memory_text(self) -> str:
    return self.memory_store.read_index_markdown()

def build_system_prompt(self) -> str:
    injected_memory = self.get_injected_memory_text()
    return (
        f"Bạn là trợ lý phát triển phần mềm cho dự án (Đang ở phiên làm việc:
{self.session_id}).\n\n"
        "=== PERSISTENT MEMORY ĐƯỢC HARNESS INJECT TỪ Ổ ĐĨA ===\n"
        f"{injected_memory}\n"
        "=======================================================\n..."
    )
```

2.4. Memory Tools (agent_tools.py)

Hai Tool được cung cấp cho Agent thông qua sự kiểm soát của Harness:

```
@tool
def save_convention_safe(topic: str, rule: str, reason: str = "Thống nhất quy ước dự
án"):
    """Lưu mới hoặc cập nhật một quy ước của dự án vào Persistent Memory thông qua
Harness."""
    return harness.save_convention(topic=topic, rule=rule, reason=reason)

@tool
def read_topic_detail_safe(topic: str):
    """Đọc chi tiết quy ước hiện hành và lịch sử các quy ước đã bị thay thế từ
memory/<topic>.md."""
    return harness.read_topic_detail(topic=topic)
```

3. Kết quả thực nghiệm và Kiểm chứng

3.1. Output khi chạy kịch bản 4 Sessions (challenge_a_demo.py)

Mỗi Session dưới đây khởi tạo một đối tượng Agent mới hoàn toàn với mảng messages đầu vào chỉ chứa duy nhất câu hỏi của phiên đó. Phần [HARNESS: INJECTED MEMORY INTO CONTEXT] được in trực tiếp để kiểm chứng dữ liệu nạp từ ổ đĩa:

```
PS D:\học tập\Agentic AI\TH7_Challenge_A> python .\challenge_a_demo.py

======================================================================
  SESSION_1 - Thiết lập quy ước dự án ban đầu (Lưu vào Persistent Memory)
======================================================================

[HARNESS: INJECTED MEMORY INTO CONTEXT]
----------------------------------------------------------------------
  # PROJECT MEMORY INDEX (MEMORY.md)
  *(Chưa có quy ước dự án nào được lưu trên ổ đĩa)*
----------------------------------------------------------------------
```

```
[USER]
  Chào bạn, chúng ta thống nhất quy ước dự án như sau nhé:
1. Chủ đề 'stack': Sử dụng PostgreSQL, TypeScript và REST API chuẩn camelCase.
2. Chủ đề 'auth': Authentication sử dụng JWT (JSON Web Token).
Hãy lưu cả 2 quy ước này vào bộ nhớ dự án.

[ACT]
  save_convention_safe({'reason': 'Thống nhất quy ước dự án', 'rule': 'PostgreSQL,
TypeScript và REST API chuẩn camelCase', 'topic': 'stack'})

[OBSERVE]
  {"status": "ok", "action": "created", "topic": "stack", "active_rule": "PostgreSQL,
TypeScript và REST API chuẩn camelCase", "superseded_old_rule": null, "updated_at":
"2026-10-22 22:06:42", "source_session": "session_1", "files_updated": ["MEMORY.md",
"memory/stack.md", "memory_store.json"]}

[ACT]
  save_convention_safe({'reason': 'Thống nhất quy ước dự án', 'rule': 'Authentication
sử dụng JWT (JSON Web Token)', 'topic': 'auth'})

[OBSERVE]
  {"status": "ok", "action": "created", "topic": "auth", "active_rule":
"Authentication sử dụng JWT (JSON Web Token)", "superseded_old_rule": null,
"updated_at": "2026-10-22 22:06:43", "source_session": "session_1", "files_updated":
["MEMORY.md", "memory/auth.md", "memory_store.json"]}

[FINAL ANSWER]
  - Đã lưu quy ước **stack**: PostgreSQL, TypeScript và REST API chuẩn camelCase.
  - Đã lưu quy ước **auth**: Authentication sử dụng JWT (JSON Web Token).
  - Các quy ước hiện hành đã được ghi vào bộ nhớ dự án và sẵn sàng sử dụng.

======================================================================
  SESSION_2 - CHỨNG MINH 1: Mở Session mới hoàn toàn - Kiểm tra Agent nhớ quy ước cũ
======================================================================

[HARNESS: INJECTED MEMORY INTO CONTEXT]
----------------------------------------------------------------------
  # PROJECT MEMORY INDEX (MEMORY.md)
  > Index ngắn chứa quy ước HIỆN HÀNH của dự án. Chi tiết lịch sử xem tại
`memory/<topic>.md`.

  - **stack**: PostgreSQL, TypeScript và REST API chuẩn camelCase (Nguồn: `session_1`,
Cập nhật: `2026-10-22 22:06:42`)
  - **auth**: Authentication sử dụng JWT (JSON Web Token) (Nguồn: `session_1`, Cập
nhật: `2026-10-22 22:06:43`)
----------------------------------------------------------------------

[USER]
  Mình vừa mở một session mới. Hãy trả lời thật ngắn gọn (tối đa 4 dòng): Dự án của
chúng ta đang dùng stack (database, ngôn ngữ, chuẩn API) gì và cơ chế authentication
nào?

[FINAL ANSWER]
```

```
  - **Stack**: PostgreSQL, TypeScript, REST API chuẩn camelCase
  - **Authentication**: Sử dụng JWT (JSON Web Token)

**********************************************************************
  CHỨNG MINH 1 (Session mới vẫn dùng đúng quy ước PostgreSQL + TypeScript + JWT): PASSED (ĐẠT)
**********************************************************************

======================================================================
  SESSION_3 - Thay đổi quy ước Authentication từ JWT sang Session Cookie
======================================================================

[HARNESS: INJECTED MEMORY INTO CONTEXT]
----------------------------------------------------------------------
  # PROJECT MEMORY INDEX (MEMORY.md)
  > Index ngắn chứa quy ước HIỆN HÀNH của dự án. Chi tiết lịch sử xem tại `memory/<topic>.md`.

  - **stack**: PostgreSQL, TypeScript và REST API chuẩn camelCase (Nguồn: `session_1`, Cập nhật: `2026-10-22 22:06:42`)
  - **auth**: Authentication sử dụng JWT (JSON Web Token) (Nguồn: `session_1`, Cập nhật: `2026-10-22 22:06:43`)
----------------------------------------------------------------------

[USER]
  Cập nhật quy ước dự án cho chủ đề 'auth': Chuyển từ JWT sang sử dụng Session Cookie (HTTP-only) để tránh rủi ro XSS. Hãy cập nhật lại bộ nhớ dự án.

[ACT]
  save_convention_safe({'reason': 'Chuyển từ JWT sang Session Cookie để tránh rủi ro XSS', 'rule': 'Sử dụng Session Cookie (HTTP-only)', 'topic': 'auth'})

[OBSERVE]
  {"status": "ok", "action": "updated_and_superseded_old", "topic": "auth", "active_rule": "Sử dụng Session Cookie (HTTP-only)", "superseded_old_rule": "Authentication sử dụng JWT (JSON Web Token)", "updated_at": "2026-10-22 22:06:47", "source_session": "session_3", "files_updated": ["MEMORY.md", "memory/auth.md", "memory_store.json"]}

[FINAL ANSWER]
  - Đã cập nhật quy ước **auth**: “Sử dụng Session Cookie (HTTP-only)”.
  - Quy ước cũ “Authentication sử dụng JWT” đã được thay thế.
  - Thay đổi đã lưu vào Persistent Memory.

======================================================================
  SESSION_4 - CHỨNG MINH 2: Mở Session mới sau khi đổi quy ước - Kiểm tra dùng quy ước mới
======================================================================

[HARNESS: INJECTED MEMORY INTO CONTEXT]
----------------------------------------------------------------------
  # PROJECT MEMORY INDEX (MEMORY.md)
  > Index ngắn chứa quy ước HIỆN HÀNH của dự án. Chi tiết lịch sử xem tại
```

```
`memory/<topic>.md`.

  - **stack**: PostgreSQL, TypeScript và REST API chuẩn camelCase (Nguồn: `session_1`,
Cập nhật: `2026-10-22 22:06:42`)
  - **auth**: Sử dụng Session Cookie (HTTP-only) (Nguồn: `session_3`, Cập nhật: `2026-
10-22 22:06:47`) *(đã thay thế 1 quy ước cũ - xem `memory/auth.md`)*
--------------------------------------------------------------------------------

[USER]
  Mình đang ở session mới (Session 4). Hãy gọi tool kiểm tra chi tiết lịch sử quy ước
'auth' và trả lời thật ngắn gọn trong 3 gạch đầu dòng:
1. Quy ước authentication HIỆN HÀNH của dự án là gì?
2. Quy ước nào ĐÃ BỊ THAY THẾ (lưu từ session nào)?
3. Khi viết API login bây giờ phải trả về gì cho client (1 dòng ngắn)?

[ACT]
  read_topic_detail_safe({'topic': 'auth'})

[OBSERVE]
  {"status": "ok", "topic": "auth", "file": "memory/auth.md", "content": "# Quy ước
chủ đề: `auth`\n\n## 1. Quy định HIỆN HÀNH (Active Convention)\n- **Nội dung**: Sử
dụng Session Cookie (HTTP-only)\n- **Lý do / Ghi chú**: Chuyển từ JWT sang Session
Cookie để tránh rủi ro XSS\n- **Nguồn (Session)**: `session_3`\n- **Thời điểm áp
dụng**: `2026-10-22 22:06:47`\n\n## 2. Lịch sử quy định ĐÃ BỊ THAY THẾ (Superseded /
Deprecated)\n- **[ĐÃ BỊ THAY THẾ #1]** `Authentication sử dụng JWT (JSON Web Token)`\n
+ Nguồn ban đầu: `session_1` (lúc `2026-10-22 22:06:43`)\n  + Đã bị thay thế bởi:
`session_3` lúc `2026-10-22 22:06:47`\n  + Lý do thay thế: Chuyển từ JWT sang Session
Cookie để tránh rủi ro XSS"}

[FINAL ANSWER]
  - **Quy ước hiện hành:** Sử dụng Session Cookie (HTTP-only) – từ `session_3`.
  - **Quy ước đã bị thay thế:** JWT authentication – được thay thế trong `session_3`
(được ghi trong `session_1`).
  - **API login:** Trả về cookie Session (HTTP-only) cho client.

**********
  CHỨNG MINH 2 (Quy ước đổi thì các session sau dùng quy ước mới Session Cookie & đánh
dấu JWT đã bị thay thế): PASSED (ĐẠT)
**********
```

3.2. Nội dung các file Memory được sinh ra trên ổ đĩa

File Index ngắn MEMORY.md:

```
# PROJECT MEMORY INDEX (MEMORY.md)
> Index ngắn chứa quy ước HIỆN HÀNH của dự án. Chi tiết lịch sử xem tại
`memory/<topic>.md`.

- **stack**: PostgreSQL, TypeScript và REST API chuẩn camelCase (Nguồn: `session_1`,
Cập nhật: `2026-10-22 22:06:42`)
- **auth**: Sử dụng Session Cookie (HTTP-only) (Nguồn: `session_3`, Cập nhật: `2026-
```

```
10-22 22:06:47`) *(đã thay thế 1 quy ước cũ - xem `memory/auth.md`)*
```

File chi tiết chủ đề memory/auth.md:

```
# Quy ước chủ đề: `auth`

## 1. Quy định HIỆN HÀNH (Active Convention)
- **Nội dung**: Sử dụng Session Cookie (HTTP-only)
- **Lý do / Ghi chú**: Chuyển từ JWT sang Session Cookie để tránh rủi ro XSS
- **Nguồn (Session)**: `session_3`
- **Thời điểm áp dụng**: `2026-10-22 22:06:47`

## 2. Lịch sử quy định ĐÃ BỊ THAY THẾ (Superseded / Deprecated)
- **[ĐÃ BỊ THAY THẾ #1]** `Authentication sử dụng JWT (JSON Web Token)`
  + Nguồn ban đầu: `session_1` (lúc `2026-10-22 22:06:43`)
  + Đã bị thay thế bởi: `session_3` lúc `2026-10-22 22:06:47`
  + Lý do thay thế: Chuyển từ JWT sang Session Cookie để tránh rủi ro XSS
```

4. Nhận xét và đánh giá

- Duy trì trí nhớ qua nhiều phiên (Chứng minh 1): Nhờ tách biệt rõ History (tin nhắn trong phiên) và Persistent Memory (quy ước lưu trên ổ đĩa), khi sang SESSION_2 với lịch sử chat rỗng, Harness chỉ cần đọc file MEMORY.md và inject vào System Prompt. Agent nắm bắt được ngay stack PostgreSQL + TypeScript + JWT mà không cần người dùng nhắc lại.
- Giải quyết xung đột khi quy ước thay đổi (Chứng minh 2): Khi người dùng đổi cơ chế xác thực sang Session Cookie ở SESSION_3, hệ thống đánh dấu JWT là đã bị thay thế kèm nguồn (session_1) và thời điểm thay thế. Sang SESSION_4, Agent áp dụng ngay quy ước mới (Session Cookie) và truy xuất được đầy đủ lịch sử thay đổi từ file memory/auth.md.
- Hiệu quả của thiết kế Index ngắn (MEMORY.md) + Topic files (memory/<topic>.md): Giúp tiết kiệm context window trong mỗi lượt gọi LLM (tránh context bloat) nhưng vẫn cho phép Agent lazy-load toàn bộ lịch sử chi tiết khi cần thiết.